Assignment No. 9.

1) Selection Sort Question.

Input 1:- [18, 22, 20, 19, 21]
output:- [18, 19, 20, 21, 22]

→

1) ~~{7, 3, 5, 2, 6} → {~~

↓1

1) {18, 22, 20, 19, 21} ⇒ {18, 22, 20, 19, 21}
unsorted

2) {18, 22, 20, 19, 21} ⇒ {18, 19, 20, 22, 21}
sorted unsorted

3) {18, 19, 20, 22, 21} ⇒ {18, 19, 20, 22, 21}
sorted unsorted

4) {18, 19, 20, 22, 21} ⇒ {18, 19, 20, 21, 22}
Sorted

5) {18, 19, 20, 21, 22} ⇒ sorted.

2) Input 2: [34, 12, 56, 45, 23]
output: [12, 23, 34, 45, 56]

→

1) [34, 12, 56, 45, 23] ⇒ {12, 34, 56, 45, 23}
unsorted

2) {12, 34, 56, 45, 23} ⇒ {12, 23, 56, 45, 34}
sorted unsorted

3] {12, 23, 56, 45, 34} ⇒ {12, 23, 34, 45, 56}
sorted unsorted

4] {12, 23, 34, 45, 56} ⇒ {12, 23, 34, 45, 56}
sorted unsorted

5] {12, 23, 34, 45, 56} ⇒ {12, 23, 34, 45, 56}
sorted unsorted

6] {12, 23, 34, 45, 56} ⇒ sorted.

- Code:-

```
int n = arr.length;

for (int i = 0; i < n-1; i++) {
    int minIndex = i;
    for (int j = i+1; j < n; j++) {
        if ( arr[j] < arr[minIndex]){
            minIndex = j;
        }
    }
    int temp = arr[i];
    arr[i] = arr[minIndex];
    arr[minIndex] = temp;
}
```

- Dry run for code input:

```
                0   1   2   3   4
input = {34, 12, 56, 45, 23}
n = 5

i = 0    0 < 4 → true
         minIndex = 0;
         j = i + 1
         j = 1    1 < 5 → true
                  arr[j] < arr[minIndex];
                  12 < 34 → true
                  minIndex = j;
minIndex = 1
                  j++
         j = 2 → 2 < 5 → true
                  arr[j] < arr[minIndex];
                  56 < 12 → false

                  j++
         j = 3 → 3 < 5 → true
                  arr[j] < arr[minIndex];
                  45 < 12 → false
                  j++
         j = 4 → 4 < 5 → true
                  arr[j] < arr[minIndex];
                  23 < 12 → false
                  j++
         j = 5    5 < 5 → false
```

temp = arr[i] → 34
arr[i] = arr[minIndex] → 12
arr[minIndex] = temp → 34

0 1 2 3 4
{ 12, 34, 56, 45, 23 }
sorted unsorted

i++

i = 1 → 1 < 5 → true.
minIndex = 1
j = 1 + 1
j = 2 → 2 < 5 → true
arr[j] < arr[minIndex];
56 < 34 → false;
j++;
j = 3 → 3 < 5 → true
arr[j] < arr[minIndex]
45 < 34 → false
j++;
j = 4 → 4 < 5 → true
arr[j] < arr[minIndex]
24 < 34 → true
minIndex = j = 4
j++;
j = 5 → 5 < 5 → false

temp = arr[i] → 34
arr[i] = arr[minIndex] → 23
arr[minIndex] = temp → 34
{ 12, 23, 56, 45, 34 }
sorted unsorted.

i++;

i = 2 2 < 5 → true.

minIndex = 2;

j = j + 1

j = 3 3 < 5 → true

arr[j] < arr[minIndex];

45 < 56 → true

minIndex = j // 3

j++

j = 4 4 < 5 → true

arr[j] < arr[minIndex];

35 < 45 → true

minIndex = 4

j++.

j = 5 5 < 5 → false.

temp = arr[i] → 56

arr[i] = arr[minIndex]; → 34

arr[minIndex] = temp → 56

{12, 23, 34, 45, 56}

sorted unsorted

i++.

i = 3 3 < 4 → true;

minIndex = 3

j = i + 1 4 < 5 → true

arr[j] < arr[minIndex];

56 < 45 → false;

j++;

j=5 5<5 → false.

temp = arr[i]; → 45
arr[i] = arr[minIndex]; → 45
arr[minIndex] = temp → 45

{12, 23, 34, 45, 56}

i++.

i = 4 4 < 4 → false.

final sorted array {12, 23, 34, 45, 56}

2) Bubble sort Question

i) Input 1: [30, 25, 27, 35, 29]
output: [25, 27, 29, 30, 35]

i) {30, 25, 27, 35, 29} ⇒ {25, 30, 27, 35, 29}
unsorted

{25, 27, 30, 35, 29} ⇐ {25, 27, 30, 35, 29}

{25, 27, 30, 29, 35}

ii) {25, 29, 30, 29, 35} ⇒ {25, 27, 30, 29, 35}
unsorted sorted

{25, 27, 29, 30, 35} ⇐ {25, 27, 30, 29, 35}

iii) {25, 27, 29, 30, 35} ⇒ {25, 27, 29, 30, 35}
unsorted sorted

{25, 27, 29, 30, 35}

iv) {25, 27, 29, 30, 35} = {25, 27, 29, 30, 35}
unsorted sorted

v) {25, 27, 29, 30, 35} ⇒ Sorted.

ii) Input 2: [15, 10, 25, 40, 30]
Output: [10, 15, 25, 30, 40]

→

1) {15, 10, 25, 40, 30} ⇒ {10, 15, 25, 40, 30}
unsorted

{10, 15, 25, 40, 30} ⇐ {10, 15, 25, 40, 30}

{10, 15, 25, 30, 40}

2) {10, 15, 25, 30, 40} ⇒ {10, 15, 25, 30, 40}
unsorted sorted

{10, 15, 25, 30, 40} ⇐ {10, 15, 25, 30, 40}

3) {10, 15, 25, 30, 40} ⇒ {10, 15, 25, 30, 40}
unsorted sorted

{10, 15, 25, 30, 40}

4) {10, 15, 25, 30, 40} ⇒ {10, 15, 25, 30, 40}
unsorted sorted

5) {10, 15, 25, 30, 40} ⇒ sorted.

Code:-

```
int n = arr.length;
for (int i = 0; i < n-1; i++) {
  for (int j=0; j < n-i-1; j++) {
    if (arr[j] > arr[j+1]) {
      int temp = arr[j];
      arr[j] = arr[j+1];
      arr[j+1] = temp;
    }
  }
}
```

Optimized Bubble sort code:-

```
for (int i=0; i < n; i++) {
    boolean swapped=false;
    for (int j=0; j < n-1-i; j++) {
      if (arr[j] > arr[j+1]) {
        int temp = arr[j];
        arr[j] = arr[j+1];
        arr[j+1] = temp;
        swapped = true;
      }
    }
    if (!swapped) {
      break;
    }
}.
```

Dry run with optimal code approach.

Input: {15, 10, 25, 40, 30}
O/p :- {10, 15, 25, 30, 40}

i = 0
↳ swapped = false;

j = 0 ... j = 1

{15, 10, 25, 40, 30} ⇒ {10, 15, 25, 40, 30}

j = 3 ... j = 2

{10, 15, 25, 40, 30} ⇐ {10, 15, 25, 40, 30}

{10, 15, 25, 30, 40}

swapped = true;

i = 1;
↳ swapped = false;

j = 0 → ~~swapped = false;~~

j = 0
↳ if → false

j = 1
↳ if → false

j = 2
↳ if → false.

if (false) → break;

3) Insertion sort Question.

i) Input: [18, 22, 20, 19, 21]
output: [18, 19, 20, 21, 22]

I) {18, 22, 20, 19, 21}
sorted unsorted.
key = 22

⇓ insert

{18, 22, 20, 19, 21}

II) {18, 22, 20, 19, 21}
sorted unsorted
key = 20

compare 22 ← left > right → 20

⇓
shift.

{18, 22, 22, 19, 21}

⇓ insert.

{18, 20, 22, 19, 21}

III) {18, 20, 22, 19, 21}
sorted unsorted.
key = 19

Compare 22 ← left > right → 19

⇓
shift

{18, 20, 22, 22, 21}

Compare 20 ← left > right → 19

⇓

shift

{18, 20, 20, 22, 21}

⇓ insert

{18, 19, 20, 22, 21}

iii {18, 19, 20, 22, 21}

sorted unsorted key = 21

Compare 22 ← left > right → 21

⇓

shift

{18, 19, 20, 22, 22}

⇓ insert

{18, 19, 20, 21, 22}

sorted.

ii) Input 2: [34, 12, 56, 45, 23]
Output: [12, 23, 34, 45, 56]

Dry run

I) {34, 12, 56, 45, 23}

sorted unsorted key = 12

compare 34 ← left > right → 12

⇓

shift.

{34, 34, 56, 45, 23}

⇓ shift

{12, 34, 56, 45, 23}

I {12, 34, 56, 45, 23}

sorted unsorted

↓ insert key = 56

{12, 34, 56, 45, 23}

II {12, 34, 56, 45, 23}

sorted unsorted

Compare 56 ← left > right → 45

↓ shift

{12, 34, 56, 56, 23}

↓ shift

{12, 34, 45, 56, 23}

IV {12, 34, 45, 56, 23}

sorted unsorted key = 23

Compare 56 ← left > right → 23

↓ shift

{12, 34, 45, 56, 56}

Compare 45 ← left > right → 23

↓ shift

{12, 34, 45, 45, 56}

Compare 34 ← left > right → 23

↓ shift

{12, 34, 34, 45, 56}

↓ insert

{12, 33, 34, 45, 56} ⇒ sorted.

- Code:-

```
int n = arr.length;
for (int i = 1; i<n; i++){
    int currentElement = arr[i];
    int left = i-1;
    while (left >= 0 && arr[left] > currentElement){
        arr[left+1] = arr[left];
        left--;
    }
    arr[left+1] = currentElement;
}
```

- Dry run for code for input

arr = {18, 22, 20, 19, 21}
(0, 1, 2, 3, 4)

int n = arr.length; // 5

~~Outer loop~~:-

i = 1; 1 < 5 → true

left = 0

currentElement = 22;

while (left >= 0 && arr[left] > currentElement)
(0)

18 > 22 → false

arr[left+1] = currentElement;

{18, 22, 20, 19, 21}

i++;

i = 2 2 < 5 → true

left = i - 1 = 2 - 1 = 1

currentElement = arr[i] // 20

while (left >= 0 && arr[left] > currentElement){
1 22 > 20

arr[left + 1] = arr[left];

{18, 20, 20, 19, 21}

left--;

}

left = 0

while (left >= 0 && arr[left] > currentElement
0 >= 0 18 > 20 → false

}

arr[left + 1] = currentElement;

{18, 20, 22, 19, 21}

i++;

i = 3 → 3 < 5 → true

left = i = 1 // 2

currentElement = arr[i] // 19

while (left >= 0 && arr[left] > currentElement){
22 > 19 → true

arr[left + 1] = arr[left];

left--;

}

{18, 20, 19, 19, 21} left = 1;

while (left >= 0 && arr[left] > currentElement){
20 > 19 → true

arr[left + 1] = arr[left];

left--;

}

{18, 20, 20, 22, 21} left = 0;

while (left >= 0 && arr[left] > currentElement)
18 > 19 → false

}
arr[left+1] = currentElement;
{18, 19, 20, 22, 21}
i++

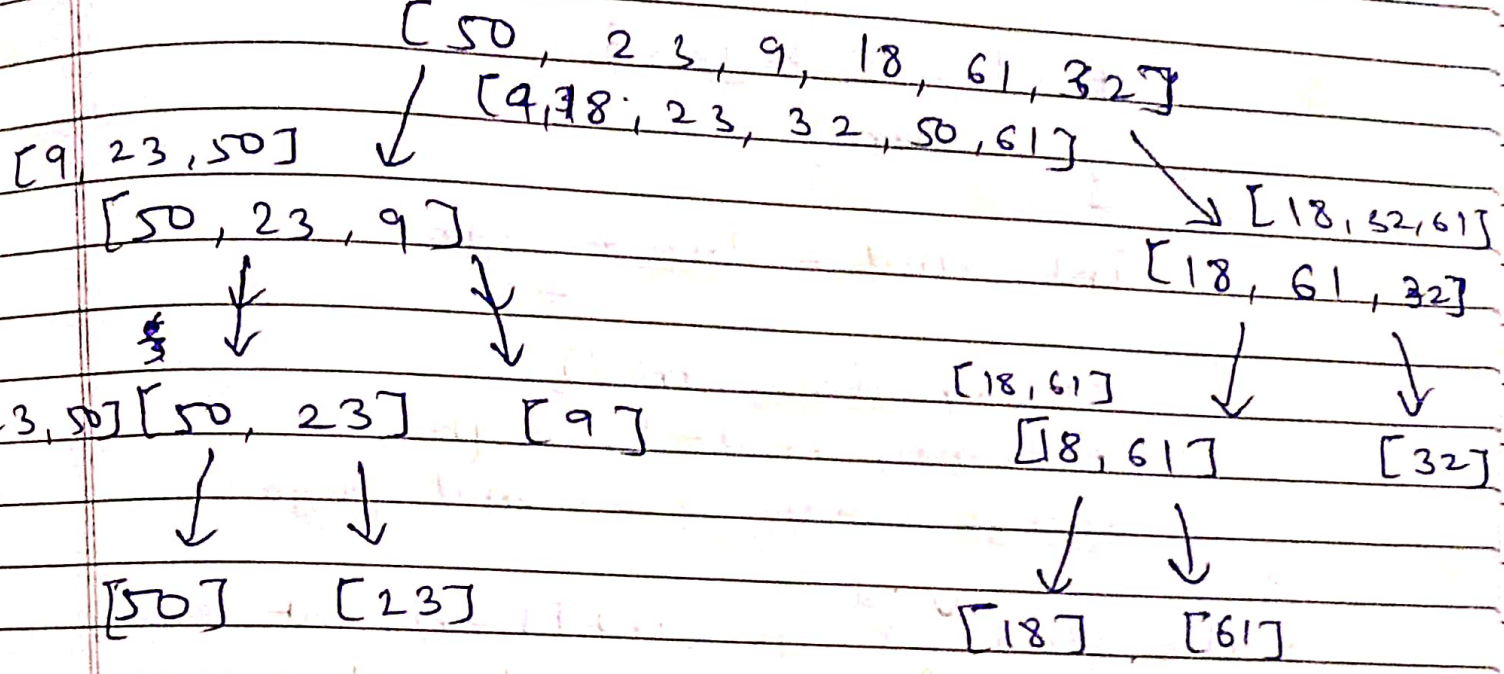
i = 4 → 4 < 5 → true
left = i - 1 // 3
currentElement = arr[i] // 21
while (left >= 0 && arr[left] > currentElement)
arr[left+1] = arr[left];
left--;
}
{18, 19, 20, 22, 22} left = 2
while (left >= 0 && arr[left] > currentElement)
20 > 21 → false
}
}
arr[left+1] = currentElement;
{18, 19, 20, 21, 22}
i++

i = 5 5 < 5 → false;

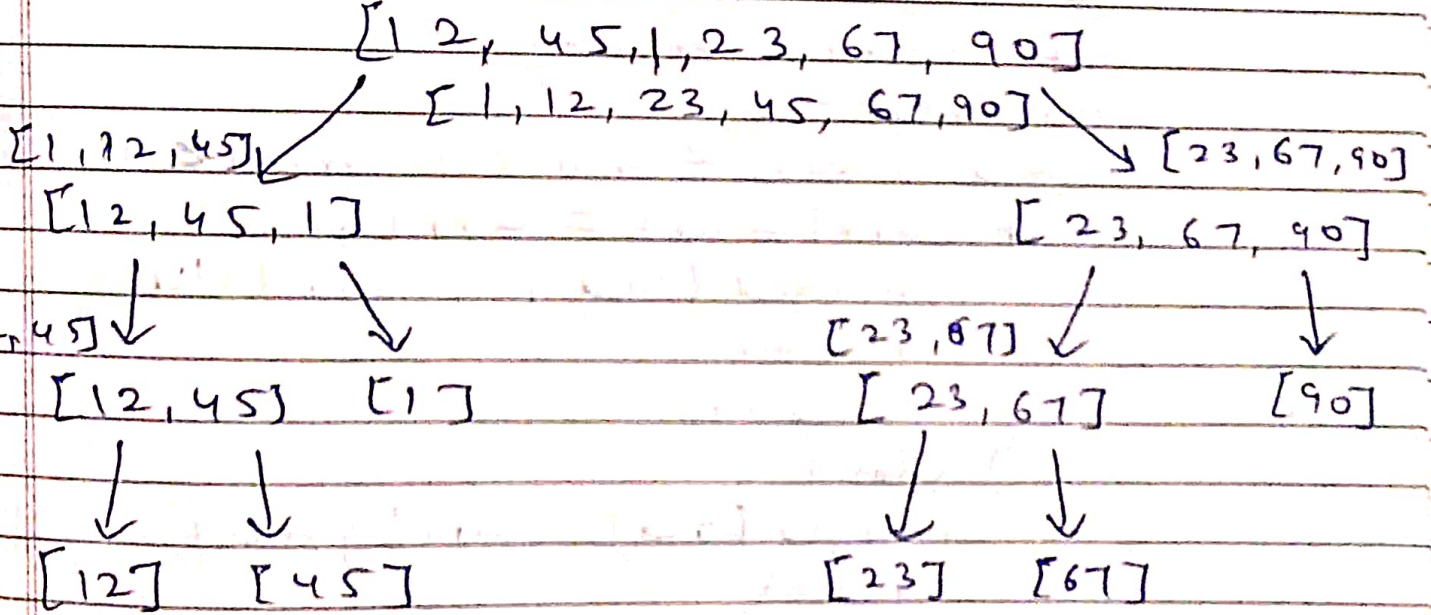
final sorted array = {18, 19, 20, 21, 22}

* merge sort.

1) [50, 23, 9, 18, 61, 32]



2) [12, 45, 1, 23, 67, 90]



- Code:-

```
public static void f(int [], arr, int low
                                    int high) {
    if (low ≥ high) {
        return;
    }
    int mid = low + high
              2
    f(arr, low, mid);
    f(arr, mid+1, high);
    merge (arr, low, mid, high);
}
public static void merge (int[] arr, int low,
                          int mid, int high) {
    int merged [] = new int [(high-low)+1];
    int blue = low;
    int green = mid+1;
    int red = 0;
    while (blue ≤ mid && green ≤ high) {
        if (arr[blue] ≤ arr[green]) {
            merged [red] = arr [blue];
            red++;
            blue++;
        } else {
            merged [red] = arr[green];
            red++;
            green++;
        }
    }
```

```
while (blue ≤ mid){
    merged [red] = arr[blue];
    red++;
    blue++;
}
while (green ≤ high){
    merged [red] = arr[green];
    red++;
    green++;
}
for(int i=0; i < merged.length; i++){
    arr[low+i] = merged[i];
}
}
```

- Dry run for input:-
  arr = {12, 45, 1, 23, 67, 30}

low = 0
high = arr.length - 1 = 5

f(arr, low, high){
(0 ≥ 5){
return;
}
mid = (high + low)/2 = (0+5)/2 = 2

f(arr, low, mid);
f(arr, mid+1, high);
merge(arr, low, mid, high);
}

```
f (arr, low, high) {
    if (0 ≥ 2) {
        return;
    f (arr, low, mid);
    f (arr, mid+1, high);
    merge (arr, low, mid, high);
}
```

mid = low + high / 2

```
f (arr, low, high) {
    if (0 ≥ 2)
        return;
    f (arr, low, mid);
    f (arr, mid+1, high);
    merge (arr, low, mid, high);
}
```

mid = low + high / 2

```
f (arr, low, high) {
    if (0 ≥ 0)
        return;
}
```

```
f (arr, low, high) {
    if (1 ≥ 1) {
        return;
    }
}
```

merge(arr, low, mid, high){

merged = [high - low + 1] = [2]

blue = 0 green = 1 red = 0

~~while (blue <= mid && green <= high){~~

0 0 1 1

12	45

arr [12, 45, 1, 23, 67, 90]

}

f(arr, low, high){

2 2

if (2 >= 2)

return;

}

merge(arr, low, mid, high){

0 1 2

12	45

1

1	12	45

arr: {1, 12, 45, 23, 67, 90}.

}

```
f(arr, low(3), high(5)) {
    mid = (high + low) / 2 = 4
    f(arr, low(3), mid(4));
    f(arr, mid+1(5), high(5));
    merge(arr, low(3), mid(4), high(5));
}

f(arr, low(3), high(4)) {
    if(3 ≥ 4)
        return;
    mid = (high + low) / 2 = (3+4)/2 = 3
    f(arr, low(3), mid(3));
    f(arr, mid+1(4), high(4));
    merge(arr, low(3), mid(3), high(4));
}

f(arr, 3, 3) {
    if(3 ≥ 3)
        return;
}

f(arr, 4, 4) {
    if(4 ≥ 4)
        return;
}
```

merge Carr, low, mid, high){
[23] & [67]
[23, 67]
arr{1, 12, 45, 23, 67, 30}
}

f Carr, ~~shoddu~~, high){
if (5 >= 5)
return;
}

merge Carr, low, mid, high){
[23, 67] & [90]
[23, 67, 90]
arr{1, 12, 45, 23, 67, 90}
}

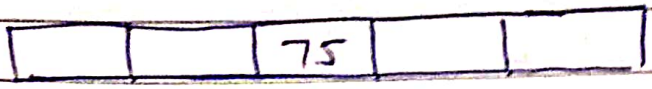
merge Carr, low, mid, high){
{10, 12, 45} {23, 67, 90}
arr{1, 12, 23, 45, 67, 90}
}

* Quick sort

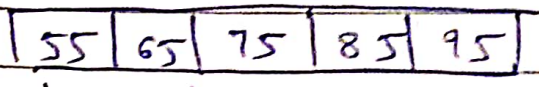
1) arr = {75, 85, 55, 95, 65}

Pivot = 75

sorted array = {55, 65, 75, 36, 95}
correct position of pivot is index 2.



Place smallest element on left side of pivot & largest in right side of pivot.



Smaller

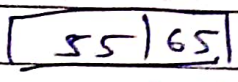
larger

{55, 65}

{85, 95}

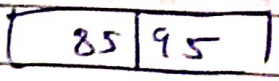
pivot = 55

pivot = 85



larger

{65}



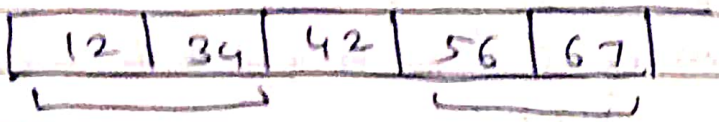
larger

{95}

2) arr {42, 56, 12, 67, 34}

Step 1 : pivot = 42

Step 2 : Sorted array {12, 34, 42, 56, 67}



smaller

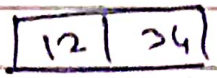
bigger

{12, 34}

{56, 67}

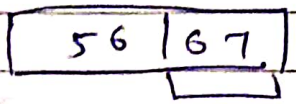
pivot = 12

pivot = 56



larger

{34}



larger

{67}

- Code :-

```
public static void f(int arr[], int low,
                                int high){
    if(low == high){
        return;
    }
    int pivot = pivotfinder(arr, low, high);
    f(arr, low, pivot-1);
    f(arr, pivot+1, high);
}
```

```
public static void pivotfinder (int [] arr,
                                int low, int high){
    int pivot = arr [low];
    int i = low;
    int j = high;
    while (i < j) {
        while (i ≤ high && arr[i] ≤ pivot){
            i++;
        }
        while (j ≥ low && arr[i] > pivot){
            j--;
        }
        if (i < j) {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[low];
    arr [low] = arr[j];
    arr [j] = temp;
    return j;
}
```